

PaperPass[免费版]AIGC检测报告

简明打印版

AIGC总体疑似度(高+中+轻): 18.7%

AIGC总体疑似度(高+中+轻): 12.93% (加权计算)

高度疑似AIGC占全文比: 1.29%
中度疑似AIGC占全文比: 16.86%
轻度疑似AIGC占全文比: 0.55%
不予检测文字占比: 78.95%

检测版本: 免费版(仅检测中文)

报告编号: ZD2E6A03F894C3BE8

论文题目: main

论文作者: 佚名

论文字数: 49087

段落个数: 182

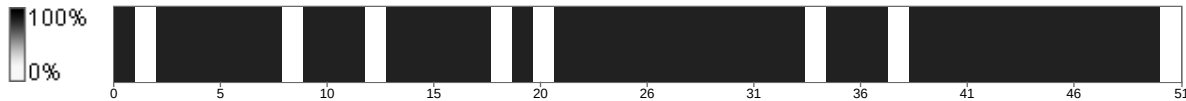
句子个数: 1124

片段个数: 51

提交时间: 2026-5-13 12:05:40

查询真伪: <https://www.paperpass.com/check>

疑似度分布图:



正文中片段的不同颜色表示不同的疑似度范围

- 红色: AIGC生成疑似度在70%以上 (高度疑似)
- 橙色: AIGC生成疑似度在60%~70% (中度疑似)
- 紫色: AIGC生成疑似度在50%~60% (轻度疑似)
- 黑色: AIGC生成疑似度在50%以下
- 灰色: 过短片段、标题和英文等不予检测的文字

注: AIGC检测可有效识别文本是否部分或全部由AI模型生成, 检测结果与论文质量无关、仅表示论文中内容片段存在AI生成可能性的概率。

加权计算公式: (片段1疑似度 + 片段2疑似度 + ... + 片段n疑似度) / n
片段疑似度范围0.0~1.0, 合格片段按照0计算, 不计算不予检测文字

AI 69%

序号: 37 学号: 1023000811 武汉理工大学课程设计(论文) 基于多技术范式的中文情感分析对比与权衡
 研究学院(系): 数学与统计学院专业班级: 统计 2302 班课程名称: 统计计算与软件学生姓名: 夏华峻指导教师: 贺紫盈摘要随着大语言模型的发展, 中文情感分析从传统统计方法进入了深度预训练阶段。但在实际应用中, 如何在 SVM、Bi-LSTM、BERT 等模型的精度和计算成本之间做取舍, 仍缺少系统的量化依据。

本文基于开源数据集 ChnSentiCorp (9,600 训练/ 1,200 验证/ 1,200 测试), 在统一硬件与软件环境下, 对比了 SVM + TF-IDF、Bi-LSTM、BERT-base 微调、Qwen2.5-7B 零样本推理和 Qwen2.5-7B LoRA 微调五类模型, 并引入早停机制保证公平。实验表明: (1)预训练是精度突破的关键——BERT 微调 ($F1=0.9521$) 比 Bi-LSTM ($F1=0.9003$) 高出 5.18 个百分点;

(2) LoRA 使 Qwen-7B 的 $F1$ 从 0.8985 跃升到 0.9570, 四项指标均为最优; (3) Qwen-LoRA 对 BERT 的精度优势仅 0.49 个百分点, 从训练时间来看, 训练成本却是 7.66 倍, 参数规模扩大的边际收益递减明显; (4) 轻量级 SVM ($F1=0.8944$, 训练 0.32s) 在计算经济性上仍有不可替代的位置。本文为中文情感分析的模型选型提供了量化参考。

关键词: 中文情感分析; 预训练语言模型; LoRA 微调; 大语言模型; 模型对比评估 | Abstract With the rise of large language models, Chinese sentiment analysis has shifted from traditional statistical methods to deep pre-training approaches. Yet in practice, there remains no systematic quantitative basis for balancing accuracy against computational cost when choosing among models such as SVM, Bi-LSTM, and BERT.

Using the open-source ChnSentiCorp dataset (9,600 training / 1,200 validation / 1,200 test), this study compares five models under uniform hardware and software: SVM + TF-IDF, Bi-LSTM, BERT-base fine-tuned, Qwen2.5-7B zero-shot, and Qwen2.5-7B LoRA fine-tuned. An early stopping mechanism was applied to ensure fair comparison. The results show: (1) pre-training is the key to the accuracy breakthrough—BERT ($F1=0.9521$) outperforms Bi-LSTM ($F1=0.9003$) by 5.18 percentage points; (2) LoRA lifts Qwen-7B from $F1=0.8985$ to 0.9570, ranking first across all four metrics; (3) Qwen-LoRA's accuracy edge over BERT is only 0.49 percentage points, yet in terms of training time, its cost is 7.66×higher, revealing significant diminishing marginal returns from parameter scaling; (4) the lightweight SVM ($F1=0.8944$, 0.32s training) remains irreplaceable in computational economy. This study provides a quantitative reference for model selection in Chinese sentiment analysis.

Key words: Chinese Sentiment Analysis; Pre-trained Language Models; LoRA Fine-tuning; Large Language Models; Model Comparative Evaluation | 目录第 1 章相关工作与文献综述.....1 1.1 传统机器学习与特征工程.....1 1.2 从序列建模到深度表示学习.....1 1.3 预训练模型与 Transformer.....2 1.4 大语言模型与参数高效微调.....2 第 2 章核心理论与模型架构.....2 2.1 基于统计特征的 SVM 分类模型.....2 2.2 基于深度序列建模的 Bi-LSTM 网络.....3 2.3 预训练编码器 BERT 模型.....3 2.4 Qwen 大规模语言模型与注意力机制.....4 2.4.1 缩放点积自注意力(Scaled Dot-Product Attention).....4 2.4.2 多头机制的并行表征(Multi-Head Mechanism).....4 2.4.3 情感分析中的全局特征聚合.....4 2.5 参数高效微调技术 LoRA.....5 第 3 章实验设计与设置.....5 3.1 数据集.....5 3.2 评价指标.....6 3.3 实验环境.....6 3.4 模型配置与超参数.....6 3.4.1 SVM + TF-IDF 基线模型.....7 3.4.2 Bi-LSTM 模型.....7 3.4.3 BERT 微调模型.....7 3.4.4 Qwen-7B 零样本推理.....7 3.4.5 Qwen-7B LoRA 微调.....7 第 4 章实验结果与分析.....8 4.1 整体性能对比.....8 4.2 训练与推理效率分析.....9 4.3 预训练范式的价值.....11 4.4 大规模语言模型分

析.....	12	4.4.1 零样本能力的边界.....	12	4.4.2 LoRA 微调的效果.....	12
果.....	12	III.4. 5 精度 - 效率权衡.....	12	4.6 综合讨论.....	13
论.....	13	第 5 章总结与展望.....	14	5.1 工作总结.....	14
结.....	14	5.2 研究局限.....	15	5.3 未来展望.....	15
望.....	15	参考文献.....	16	附录 A 实验代码.....	17
码.....	17	附 A1 SVM + TF-IDF 基线模型.....	17	附 A2 Bi-LSTM 序列分类模型.....	19
序.....	19	附 A3 BERT -base 预训练微调模型.....	22	附 A4 Qwen2.5-7B 零样本推理.....	24
列.....	24	附 A5 Qwen2.5-7B LoRA 微调模型.....	26	附 A6 论文图表可视化脚本.....	29
分.....	26	IV 第 1 章相关工作与文献综述			
类.....	29	文本情感分析 (Sentiment Analysis)			
型.....		旨在从文本中自动抽取主观态度信息，是自然语言处理 (NLP) 的基础任务之一。随着社交平台与电商评论的规模不断膨胀，该技术已被广泛用于舆情监控、产品推荐等实际场景。从方法演进的角度看，情感分析经历了从手工规则、统计机器学习、深度学习，再到大语言模型的四个阶段。			

AI 70%

图 1.1 中文情感分析技术发展脉络 1.1 传统机器学习与特征工程在统计 NLP 的早期，研究者关注如何从文本中构造有区分力的特征。Joachims[1]论证了 SVM 在处理高维稀疏文本时的优势——文本特征维度远高于样本量，而 SVM 通过寻找最大间隔超平面来抑制过拟合。针对中文，Tan 等人[2]在 ChnSentiCorp 等语料上发现，分词质量和特征选择方法（如 TF-IDF）对最终精度影响很大。但这类方法将文本视为词袋，丢掉了词序，遇到否定、反讽等需要通过上下文消歧的情况就容易出错。

AI 70%

1.2 从序列建模到深度表示学习 Word2Vec[3]把词映射为低维稠密向量，使语义计算成为可能。Kim[4]用多尺度 CNN 做句子分类，验证了局部 n-gram 特征的有效性。情感分析依赖长程上下文，因此 LSTM[5]及其双向版本（Bi-LSTM）受到更多关注。Long 等人[6]在 Bi-LSTM 上引入多头注意力，证明双向结构能同时编码前向与后向语义，提升了对长句情感逻辑的建模能力。不过 RNN 类模型是串行计算的，训练并行度低，当文本超过百词时梯度回传仍然吃力。

AI 70%

1.3 预训练模型与 Transformer Vaswani 等人[7]提出自注意力机制，让任意两个词之间的信息通路不再受距离限制。

Devlin 等人[8]在此基础上设计了 BERT：用掩码语言模型在海量文本上预训练，再在下游标注数据上微调。BERT 及其改进版 RoBERTa[9]在情感分析等分类任务中长期占据最优位置。

AI 70%

1.4 大语言模型与参数高效微调以 GPT 系列为代表的生成式大模型在少样本和零样本场景下展现出可观的泛化能力。Brown 等人[10]探索了上下文学习 (In-Context Learning) 的潜力；Edwards 与 CamachoCollados[11]在 16 个分类数据集上做了系统对比，结论是：在标注数据可得的前提下，微调小模型往往优于大模型的少样本提示。出于算力约束，参数高效微调（PEFT）日益受到重视。Hu 等人[12]提出的 LoRA 在冻结预训练权重的同时插入低秩旁路矩阵，将可训练参数量压缩到原模型的千分之一量级，使得在单卡 GPU 上微调 7B 13B 规模模型（如 Qwen 系列、LLaMA[13]）变得可行。

AI 50%

综合来看，各技术路线的精度在持续提高，但工程上选择哪种模型不能只看精度。本研究的出发点就是：在统一的实验条件下，对 SVM、Bi-LSTM、BERT、Qwen 等代表性方法做横向对比，为模型选型提供量化的参考。

AI 70%

第 2 章核心理论与模型架构本章介绍实验中用到的五种模型，涵盖统计机器学习、深度序列建模、预训练编码器和 大语言模型四个阶段。各模型在特征提取方式、上下文建模能力和参数更新策略上各有不同。

2.1 基于统计特征的 SVM 分类模型在深度学习流行之前，支持向量机（Support Vector Machine, SVM）一直是文本分类的标准方案。文本通过词袋模型（Bag-of-Words）转为高维稀疏向量后，送入 SVM 寻找最大间隔超平面进行分类。

为抑制高频停用词的影响，本实验用 TF-IDF（Term Frequency-Inverse Document Frequency）对文本做特

征加权。词 t 在文档 d 中的 TF-IDF 权重为：

$TF\text{-}IDF(t, d) = TF(t, d) \times \log(N / DF(t) + 1)$ (2.1) 其中， $TF(t, d)$ 是词频， N 是文档总数， $DF(t)$ 是包含该词的文档数。这个公式会压低“的”“了”等高频词的权重，抬高有情感区分度的特征词。

SVM 在这个 TF-IDF 空间中求解最大间隔超平面。引入软间隔和惩罚参数 C 后，对偶优化问题为：

$\min \alpha \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(x_i, x_j) - m \sum_{i=1}^m \alpha_i$ s.t. $\sum_{i=1}^m \alpha_i y_i = 0, 0 \leq \alpha_i \leq C$ (2.2) α_i 是拉格朗日乘子， $K(x_i, x_j)$ 是核函数。文本特征通常近似线性可分，用线性核的 SVM 在特征数远大于样本数时泛化能力很好，且优化目标为凸二次规划，收敛速度快。

AI 70%

2.2 基于深度序列建模的 Bi-LSTM 网络词袋模型丢掉了词序。但中文情感经常依赖前后文的逻辑关系——比如双重否定、转折词。长短期记忆网络 (LSTM) 通过门控机制缓解了 RNN 的梯度消失和爆炸问题。

LSTM 单元包含遗忘门 f_t 、输入门 i_t 和输出门 o_t 。在时间步 t ，输入词嵌入向量 x_t ，细胞状态 C_t 和隐藏状态 h_t 的更新为：

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ (2.3) $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ (2.4) $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$ (2.5) $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ (2.6) $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ (2.7) $h_t = o_t \odot \tanh(C_t)$ (2.8) 单向 LSTM 只看到一个方向的信息。中文评论的转折词常出现在句中甚至句末，只靠前向编码会丢失后文信息。本实验用双向 LSTM (Bi-LSTM)，把前向隐藏状态 $\rightarrow h_t$ 和后向隐藏状态 $\leftarrow h_t$ 拼起来：

$h_t = [\rightarrow h_t \oplus \leftarrow h_t]$ (2.9) 双向结构使每个词都能同时感知左右两侧的语境，有助于捕捉中文句法和情感翻转。

2.3 预训练编码器 BERT 模型 Transformer 不再依赖循环时序，而是通过自注意力 (Self-Attention) 实现并行计算和全局上下文建模。BERT (Bidirectional Encoder Representations from Transformers) 基于多层双向 Transformer 编码器构建，其核心操作为缩放点积自注意力：

AI 69%

$Attention(Q, K, V) = \text{softmax}(QK^T \sqrt{d_k})V$ (2.10) 这部分数学原理在第 2.4 节展开。BERT 通过掩码语言模型 (Masked Language Model, MLM) 在中文语料上双向预训练，获得了深层上下文语义表征。微调时，在输入开头放一个 [CLS] 标记，经多层自注意力交互后，取该标记的最终隐状态 $C \in \mathbb{R}^H$ 作为整句语义表示，送入线性分类层，用交叉熵损失优化。

AI 70%

2.4 Qwen 大规模语言模型与注意力机制 本实验还引入了通义千问 (Qwen)。Qwen 是阿里云发布的开源大模型系列，基于 Transformer 解码器 (Decoder-only) 架构，采用因果自注意力 (Causal Self-Attention) 逐 Token 做自回归预测。实验选用 Qwen-7B：70 亿参数、32 层 Transformer、32 注意力头、隐藏维度 4096，规模远大于 BERT-base (约 1.1 亿参数)。

2.4.1 缩放点积自注意力 (Scaled Dot-Product Attention) 自注意力的本质是计算序列中每对 Token 之间的相关性。输入向量序列通过三个可学习权重矩阵映射为查询向量 Q 、键向量 K 、值向量 V ：

$Q = XW_Q, K = XW_K, V = XW_V$ (2.11) 其中， $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 。用缩放点积计算 Q 和 K 的相似度：

$Attention(Q, K, V) = \text{softmax}(QK^T \sqrt{d_k})V$ (2.12) 除以 $1/\sqrt{d_k}$ 是为了防止点积值过大，导致 Softmax 进入梯度平坦区。

AI 70%

2.4.2 多头机制的并行表征 (Multi-Head Mechanism) 单头注意力只在一个子空间内建模。多头机制把 Q, K, V 投影到 h 个不同的低维子空间，并行计算。比如在做情感分析时，一个头可能关注否定词，另一个关注转折连词。

AI 68%

对第 i 个头：

$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$ (2.13) 所有头的输出拼接后经过 WO 做线性变换：

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)WO$ (2.14) 2.4.3 情感分析中的全局特征聚合在情感分类中，全连接的注意力图消除了序列距离的限制。比如输入“虽然环境很差，但味道真的一流”，MHSA 让句末的“一流”能直接和句首“但”转折后的语义建立关联，从而定位正向信号。这是 RNN 和浅层模型做不到的全局依赖建模。

AI 66%

42.5 参数高效微调技术 LoRA Qwen-7B 虽然在零样本下已具备一定的判断力，但对 70 亿参数做全量微调的显存和计算开销远超常规实验条件。

LoRA (Low-Rank Adaptation) 的思路是：大模型适配下游任务时，权重更新量 ΔW 具有低秩性质。因此可以在冻结主干权重 $W_0 \in \mathbb{R}^{d \times k}$ 的同时，旁路插入低秩分解矩阵：

$h = W_0 x + \Delta W x = W_0 x + B A x$ (2.15) $B \in \mathbb{R}^{d \times r}$ 和 $A \in \mathbb{R}^{r \times k}$ 是可训练的低秩矩阵， $r \ll \min(d, k)$ 。反向传播只更新 A 和 B 的梯度。本实验取 $r = 16$ ，可训练参数压缩到全量微调的千分之几。在单张 GPU 上即可完成 7B 级别模型的情感分类适配，推理时做权重合并，延迟与原模型一致。

AI 70%

第 3 章实验设计与设置本章从数据集、评价指标、实验环境及超参数配置四个方面说明实验的设计。

3.1 数据集实验数据来自 HuggingFace 开源数据集 linsinute/ChnSentiCorp，包含酒店、笔记本电脑、书籍等场景的中文用户评论，每条标注为正向 (1) 或负向 (0)。这是一个在中文情感分析中常用的基准数据集。

AI 69%

原始数据以 Parquet 格式存储，划分为 train (9,600 条)、validation (1,200 条) 和 test (1,200 条) 三个子集。实验中，SVM 将训练集与验证集合并为一（共 10,800 条）进行训练，不设独立验证集；Bi-LSTM、BERT 与 Qwen-LoRA 使用训练集 (9,600 条) 训练，验证集 (1,200 条) 用于超参数监控与最优模型选择。所有模型在测试集 (1,200 条) 上评估。具体划分如表 3.1 所示。

表 3.1 ChnSentiCorp 数据集划分子集 SVM Bi-LSTM BERT/Qwen-LoRA 训练集 10,800 9,600 9,600 验证集— 1,200 1,200 测试集 1,200 1,200 1,200 总计 12,000 12,000 12,000 在预处理阶段，所有文本经过中文分词 (SVM 与 Bi-LSTM 使用 Jieba 分词，BERT 与 Qwen 使用各自的 Tokenizer)。

53.2 评价指标情感分类本质上是一个二分类问题。本文采用准确率 (Accuracy)、精确率 (Precision)、召回率 (Recall) 和 F1-Score 四项指标进行评估：

AI 68%

$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$ (3.1) $Precision = \frac{TP}{TP + FP}$ (3.2) $Recall = \frac{TP}{TP + FN}$ (3.3) $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$ (3.4) 其中，TP 为真正例，TN 为真负例，FP 为假正例，FN 为假负例。准确率反映整体正确率；精确率衡量模型判为正类时的可靠性；召回率衡量对正类样本的覆盖率；F1-Score 是精确率与召回率的调和平均，用于综合评价分类质量。此外，实验同时记录了各模型的训练耗时和推理耗时。

3.3 实验环境所有模型的训练和测试在统一的硬件与软件环境下完成，配置如表 3.2。

表 3.2 实验软硬件环境组件配置 GPU NVIDIA RTX PRO 6000 Blackwell Server Edition (96 GB VRAM) CPU Intel(R) Xeon (R) Platinum 8470Q 内存 110 GB 操作系统 Ubuntu 22.04 Python 3.12 PyTorch 2.11.0 + CUDA 12.8 Transformers 5.8.0 Scikit-learn 1.8.0 3.4 模型配置与超参数为保证实验公平，各模型的超参数在预实验调优后确定，全程保持不变。

AI 69%

63.4.1 SVM + TF-IDF 基线模型使用 TF-IDF 向量化器对分词后的文本做特征提取，最大特征数设为 10,000。分类器选用 LinearSVC，惩罚系数 $C = 1.0$ ，最大迭代次数 2,000，其余参数为 Scikit-learn 默认值。

AI 70%

3.4.2 Bi-LSTM 模型 Bi-LSTM 从零学习词嵌入。词表大小 15,000 (含 <PAD> 和 <UNK> 两个特殊标记)，嵌入维度与隐藏层维度均为 128。LSTM 采用单层双向结构，在分类层前施加 Dropout (比率 0.5)。优化器为 Adam，初始学习率 1×10^{-3} ，批次大小 64。训练引入早停机制：最大训练轮数 50，验证集损失连续 5 轮不下降则终止训练，取验证集准确率最高的检查点作为最终模型。

AI 70%

3.4.3 BERT 微调模型模型采用 bert-base-chinese 预训练权重：12 层 Transformer 编码器，隐藏维度 768，总参数量约 1.1 亿。微调使用 HuggingFace Trainer 框架，学习率 2×10^{-5} ，批次大小 32，最大序列长度 128。训练引入早停机制 (EarlyStoppingCallback)：最大训练轮数 20，耐心值 3，验证集指标连续 3 轮不提升则终止。预热步数比例 (Warmup Step Ratio) 为 0.1，权重衰减为 0.01。每轮评估并保存检查点，训练结束后加载验证集最优检查点。

AI 69%

3.4.4 Qwen-7B 零样本推理零样本实验使用 Qwen2.5-7B 预训练权重，以 BF16 精度加载，通过 device_map="auto" 自动分配至 GPU。该模型拥有 70 亿参数、32 层 Transformer 解码器、32 个注意力头。推

理时采用 Chat Template: 系统消息为"情感分析: 好评输出 1, 差评输出 0。", 用户消息为待分析文本, 随后通过规则提取标签 (检测输出中是否含'1'或'正'判为正向, '0'或'负'判为负向)。最大生成 Token 数 5, 关闭随机采样 (do_sample=False)。此实验不更新模型权重。

3.4.5 Qwen-7B LoRA 微调 LoRA 微调同样基于 Qwen2.5-7B 预训练权重, 以 BF16 精度加载。模型头部替换为二分类的 AutoModelForSequenceClassification 结构, 设 pad_token_id 为 eos_token_id。

AI 69%

LoRA 配置: 秩 $r=16$, 缩放因子 $\alpha=32$, Dropout 比率 0.05, 目标模块为注意力层的 Query 投影 (q_proj) 和 Value 投影 (v_proj)。训练时冻结原始模型全部权重, 仅优化 LoRA 新增参数。

AI 66%

7 训练参数: 批次大小 16, 梯度累积步数 2 (等效批次 32), 学习率 1×10^{-4} , 预热步数比例 0.05。引入早停机制: 最大训练轮数 20, 耐心值 3。启用 BF16 混合精度训练, 每轮评估并保存检查点, 结束加载最优检查点。

AI 68%

第 4 章实验结果与分析本章从分类精度、计算效率、预训练增益、大语言模型表现以及精度-效率权衡五个角度分析实验结果。所有数据均在第 3 节所述的统一实验设置下运行一次获得。

4.1 整体性能对比五模型在测试集上的综合性能如表 4.1、图 4.1 与雷达图 4.3 所示。

表 4.1 五模型整体性能对比模型 Accuracy Precision Recall F1-Score SVM + TF-IDF 0.8933 0.8974 0.8914 0.8944 Bi-LSTM 0.8983 0.8945 0.9062 0.9003 BERT (Fine-tune) 0.9517 0.9568 0.9474 0.9521 Qwen-7B (Zero) 0.9000 0.851 0.8734 0.8985 Qwen-7B (LoRA) 0.9567 0.9633 0.9507 0.9570 SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 0.8800.9000.9200.9400.960 0.8933 0.8983 0.9517 0.9000 0.9567 0.8944 0.9003 0.9521 0.8985 0.9570 Best Accuracy F1-Score 图 4.1 五模型 Accuracy 与 F1-Score 对比按 F1-Score 排序: Qwen-LoRA (0.9570) > BERT (0.9521) > Bi-LSTM (0.9003) > QwenZero (0.8985) > SVM (0.8944)。Qwen-LoRA 在四项指标上均为最优, 说明借助 LoRA 和充分的训练轮数, 大模型的潜力得到了有效释放。BERT 虽然只有 1.1 亿参数, F1 仅比 QwenLoRA 低 0.49 个百分点, 显示出预训练编码器在这类任务上的竞争力。Bi-LSTM 加入早停后 F1 提升到 0.9003, 超过了 SVM 和 Qwen-Zero, 也说明即使结构简单的模型, 配合适当的正则化同样能得到不错的成绩。SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 0.880.890.900.910.920.930.940.950.96 0.8944 0.9003 0.9521 0.8985 0.9570 +0.0059 +0.0518 -0.0536 +0.0585 SVM baseline (0.8944)

AI 70%

图 4.2 F1-Score 增量瀑布图 (以 SVM 为基线) 图 4.2 的增量瀑布图给出了以 SVM 为基线的各阶段性能递进: SVM $\rightarrow$ Bi-LSTM 的增益很小 (+0.0059), 再 $\rightarrow$ BERT 则有大幅跃升 (+0.0518, 增幅 5.75%), 说明预训练是其间的转折; Qwen-Zero $\rightarrow$ Qwen-LoRA 的增益为 +0.0585, 显示 LoRA 对零样本能力的转化效果显著。

AI 70%

图 4.3 的雷达图展示了各模型在四维指标上的归一化轮廓。Qwen-LoRA 在所有方向上均是最外层, BERT 紧挨其后。SVM 和 Bi-LSTM 的轮廓接近, Qwen-Zero 在 Recall 方向收缩明显 (0.8734), 与前述零样本推理偏保守的判断一致。

AI 70%

4.2 训练与推理效率分析精度之外, 计算开销是工程选型时不可忽略的维度。图 4.4 给出了各模型的训练耗时和测试耗时。

训练方面, SVM 只需 0.32 秒, Bi-LSTM 为 4.05 秒 (早停于第 9 轮), 均属轻量级, 适合快速迭代和超参数搜索。BERT 的训练耗时 184.66 秒 (约 3.1 分钟, 早停于第 8 轮), 处于中等水平。Qwen-LoRA 训练耗时最长, 为 1414.10 秒 (约 23.6 分钟, 早停于第 10 轮), 分别是 BERT 的 7.66 倍和 Bi-LSTM 的 349 倍。考虑到 LoRA 只增加了极少比例的可训练参数, 这一开销在工程上仍然可控。

9 Accuracy Precision Recall F1-Score 0.8900.9100.9300.9500.970 SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 图 4.3 五模型四维性能雷达图 SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 100101102103 0.3s 4.0s 184.7s 0.0s 1414.1s 10 210 1100101102 0.02s 0.02s 1.06s 43.84s 7.19s 图 4.4 五模型训练与推理耗时双轴对比 10 推理方面, SVM 和 Bi-LSTM 均为 0.02 秒, 吞吐量约 60,000 条/秒, 可以在普通 CPU 环境下部署。BERT

推理耗时 1.06 秒，也属可接受范围。Qwen-LoRA 推理 7.19 秒，为 BERT 的 6.78 倍。Qwen-Zero 的推理最慢，43.84 秒处理 1,200 条样本 (58.03 Tokens/s)，大规模上线时会有较大的延迟压力。得益于 RTX PRO 6000 的加速，Qwen 系列推理速度较之前已有明显提升。

AI 70%

4.3 预训练范式的价值 Bi-LSTM 和 BERT 之间最值得关注的对比是：两者都从标注数据中学习，但 BERT 由于受益于 MLM 预训练，表现大幅领先。

具体来看，BERT 的 Accuracy (0.9517) 比 Bi-LSTM (0.8983) 高 5.34 个百分点，F1 高 5.18 个百分点。原因在于 Bi-LSTM 只能在 9,600 条标注样本上学到表征，而 BERT 已在海量中文语料上通过预训练积累了丰富的语义知识。在标注数据受限的情况下，预训练注入的先验信息对性能的决定性最大。

从 Precision-Recall 角度看 (图 4.5)，Bi-LSTM 呈现出“低精高召”：(Precision 0.8945, Recall 0.9062)，它倾向于做积极判断，但误报更多。BERT 的 Precision 和 Recall 更均衡 (0.9568 和 0.9474)，分类既准也全——这在舆情监控等对误报敏感的场景中更有优势。

	SVM	Bi-LSTM	BERT	Qwen Zero-shot	Qwen LoRA
Accuracy	0.8600	0.8800	0.9000	0.9200	0.9400
Precision	0.8974	0.8945	0.9568	0.9251	0.9633
Recall	0.8914	0.9062	0.9474	0.8734	0.9507
P-R=+0.0060	P-R=-0.0117	P-R=+0.0094	P-R=+0.0517	P-R=+0.0126	

图 4.5 五模型 Precision 与 Recall 对比 114.4 大规模语言模型分析 4.4.1 零样本能力的边界 Qwen-7B 在零样本设置下 Accuracy 为 0.9000，略高于 Bi-LSTM (0.8983) 和 SVM (0.8933)，但优势很小——差距仅 0.17 个百分点。这说明在 ChnSentiCorp 这种情感信号比较显式的数据集上，70 亿参数的零样本推理能力并不比精心设计的特征工程 (TF-IDF + SVM) 更强。大模型的长处——常识推理、复杂语义理解——在简单的二分类任务中体现不出来。

此外，Qwen-Zero 的 Recall (0.8734) 在五模型中最低，表明它在零样本条件下偏保守，容易漏掉部分正样本。

4.4.2 LoRA 微调的效果加 LoRA 微调后，Qwen-7B 的 Accuracy 从 0.9000 提升到 0.9567 (+5.67%)，F1 从 0.8985 提升到 0.9570 (+5.85%)，并在四项指标上全部超过 BERT，在本次实验中成为最优模型。

AI 67%

LoRA 仅引入 $r=16$ 的低秩矩阵，只适配了 Query 和 Value 两个投影模块，未改动模型主体，却有效地把通用语义知识适配到了领域情感分类中。

Qwen-LoRA 超越 BERT 的原因可以从几个方面理解：首先，Qwen-7B 在预训练过程中积累的语义知识远超 BERT (1.1 亿参数 vs 70 亿)，虽然 Decoder-only 架构在分类任务上不如双向编码器“顺手”，但知识量的优势部分弥补了架构差异。其次，LoRA 在秩仅为 16、只适配两个投影层的保守配置下就取得了最优成绩，说明低秩假设在情感分类上成立。进一步扩大适配模块范围 (如加上 Key 和 Output 投影)，或者增大秩，还有继续提升的空间。

4.5 精度-效率权衡图 4.6 以散点图展示五模型在 F1 与训练时间两个维度上的分布，可以从中看到一条明确的“帕累托前沿”：

AI 56%

- SVM 位于左下角——训练成本极低，精度中等，是成本敏感场景的下限参考。

- BERT 处于中段——训练成本是 SVM 的 577 倍 (184.66 s / 0.32 s)，但 F1 提高了 5.77 个百分点，单位算力的精度回报最高。

AI 70%

- Qwen-LoRA 在最右上角——F1 最高 (0.9570)，但训练成本是 BERT 的 7.66 倍，推理成本是 BERT 的 6.78 倍。它代表了精度优先的选择，BERT 则在效率上更优。

- Qwen-Zero——零训练成本，精度与 Bi-LSTM、SVM 相当，但推理慢，适合完全没有标注数据且对延迟不敏感的场景。

12100 101 102 103 Training Time (s, log scale) 0.880.890.900.910.920.930.940.950. 96 Pareto Frontier SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 图 4.6 模型精度-训练效率权衡散点图如果以“单位训练时间的 F1 增益”作为效率指标，BERT 仍是最经济的选择——它以适中的训练开销换到了接近最优的精度。Qwen-LoRA

的增量计算开销在当前数据集规模下只换来了 0.49 个百分点的 F1 提升，边际回报有限。

图 4.7 从参数量维度印证了上述判断：模型参数从零（SVM）增长到 BERT 的 1.1 亿，F1 从 0.8944 升到 0.9521；但从 1.1 亿再增长到 Qwen-7B 的 70 亿，F1 仅再提高 0.49 个百分点。参数规模与性能之间的边际递减规律在图中表现得很清楚。

4.6 综合讨论综上所述：

(1) 预训练是这次实验中精度提升的最大单一因素。SVM 到 BERT 的 F1 跃升 (0.8944→0.9521, +5.77%) 远远大于模型规模扩大带来的收益，无监督预训练对通用语义表征的作用是决定性的。

(2) 大模型微调有精度上限优势，但效率代价明显。在当前设定下，Qwen-LoRA (F1=0.9570) 比 BERT (F1=0.9521) 只多了 0.49 个百分点，训练代价却高出数倍。这表明当标注数据约 1 万条时，中等规模模型配合充分的预训练已经接近性能上限，继续扩大模型规模的性价比不高。

(3) 零样本能力的表现依赖于任务本身。Qwen-7B 零样本 F1 (0.8985) 与 SVM (0.8944)、Bi-LSTM (0.9003) 处于同一水平面。在简单、定义明确的任务上，大模型的零样本优势有限。

130 100 101 102 103 104 Model Parameters (Millions, log scale) 0.890.900.910.920. 930.940.950.96 SVM Bi-LSTM BERT Qwen Zero-shot Qwen LoRA 图 4.7 模型参数规模与 F1-Score 关系气泡图 (4) LoRA 是高效且经过验证的适配方法。LoRA 以极低参数量实现了 Qwen 性能的大幅提升 (F1 +5.85%)，在 $r=16$ 、仅适配 Query 和 Value 投影的保守配置下取得了最佳 F1。增加适配模块或增大秩有可能带来进一步的增益。

以上结果为中文情感分析的模型选型提供了参考：在标注数据充足且对实时性要求高的场景，轻量级 SVM 仍有价值；在约 1 万条标注数据的典型场景，BERT 微调在精度和效率之间达到了最佳平衡；Qwen-LoRA 则在精度优先时是最优选择。两者间的 0.49 个百分点 F1 差距是否值得投入更多算力，取决于具体业务对精度的要求。

第 5 章总结与展望 5.1 工作总结本文在 ChnSentiCorp 数据集上对 SVM + TF-IDF、Bi-LSTM、BERT-base 微调、Qwen2.57B 零样本和 Qwen2.5-7B LoRA 微调这五类模型进行了横向对比。实验在统一环境与超参数下完成，引入了早停机制确保公平。主要结论：

(1) 预训练是精度提升的最大推动力。从 Bi-LSTM (F1=0.9003) 到 BERT (F1=0.9521)，预训练范式带来了 5.18 个百分点的 F1 增益，占实验全程性能提升的大部分。在大规模无标注语料上积累的通用语义知识，是下游小样本任务的质量基石。

(2) LoRA 高效释放了大模型的潜力。Qwen2.5-7B 经 LoRA 微调后，F1 从 0.8985 升至 0.9570 (+5.85%)，在全部四项指标上达到最优。以极低的参数量实现了显著的性能跃迁，验证了 PEFT 技术路线的有效性。

(3) 精度-效率的权衡清晰可见。Qwen-LoRA 以 7.66 倍于 BERT 的训练成本换取 0.49 个百分点的 F1 提升——边际收益递减明显。在约 1 万条标注数据的条件下，BERT 是精度与效率的最佳平衡点。

(4) 传统方法和零样本在特定场景仍有意义。SVM (F1=0.8944, 训练 0.32s) 的计算经济性无可替代；Qwen-Zero (F1=0.8985) 零训练成本给出有监督浅层模型级别的精度，适合标注完全不可得的场景。

5.2 研究局限 (1) 数据集限制。实验限于 ChnSentiCorp 酒店评论语料，情感表达比较直接。结论能否推广到金融、医疗等垂直领域，或长文本、多分类任务，需要更多数据验证。

(2) LoRA 配置不够丰富。目前只在 $r=16$ 、适配 Query 和 Value 的配置下实验。更优的秩、更广的模块选择（如增加 Key 和 Output 投影）、更多的标注数据，都有可能进一步拉开 Qwen-LoRA 与 BERT 的差距。

(3) 推理评估不够细。只统计了端到端推理耗时，没有区分分词、KV-Cache 和矩阵运算各自的占比，也没有测试批量推理 (Batch Inference) 的吞吐表现。

(4) 零样本 Prompt 未充分探索。只用了固定的一套提示词，没有尝试不同 Prompt 模板对 Qwen-Zero 精度的潜在提升。

5.3 未来展望 (1) 跨领域验证。在电商、社交媒体、金融评论等多领域语料上做统一对比，检验各模型在

不同文本分布和情感复杂度下的表现稳定性。

(2) LoRA 超参数搜索。对秩 r (8, 16, 32, 64)、适配模块组合 (Q/K/V/O 各种排列)、学习率和 Dropout 率做网格搜索, 找出情感分类任务上的最优 LoRA 配置。

AI 69%

(3) Prompt 和上下文学习优化。系统比较多组 Prompt 模板下的零样本和少样本 (Fewshot) 表现, 考察 In-Context Learning 在不动模型权重的前提下能拉近多少与大模型微调的差距。

(4) 推理加速探索。尝试模型量化 (4-bit/8-bit)、FlashAttention 和 vLLM 推理框架, 看能否缩小大模型部署与轻量模型之间的效率差距。

15 参考文献[1]JOACHIMS T. Text categorization with support vector machines: Learning with many relevant features[C]//European conference on machine learning. 1998: 137-142.

[2]TAN S, ZHANG J. An empirical study of sentiment analysis for chinese documents[J]. Expert Systems with applications, 2008, 34(4): 2622-2629.

[3]MIKOLOV T, et al. Efficient estimation of word representations in vector space[A]. 2013.

[4]KIM Y. Convolutional neural networks for sentence classification[C]//Proceedings of the 2014 EMNLP. 2014: 1746-1751.

[5]HOCHREITER S, SCHMIDHUBER J. Long short-term memory[J]. Neural computation, 1997.

[6]LONG F, ZHOU K, OU W. Sentiment analysis of text based on bidirectional lstm with multi-head attention[J]. IEEE Access, 2019, 7: 141960-141969.

[7]VASWANI A, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017.

[8]DEVLIN J, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[A]. 2018.

[9]LIU Y, et al. Roberta: A robustly optimized bert pretraining approach[A]. 2019.

[10]BROWN T, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020.

[11]EDWARDS A, et al. Language models for text classification: Is in-context learning enough? [C]//Proceedings of the LREC-COLING 2024. 2024: 10058-10072.

[12]HU E J, et al. Lora: Low-rank adaptation of large language models[A]. 2021.

[13]TOUVRON H, LAVRIL T, IZACARD G, et al. Llama: Open and efficient foundation language models[A]. 2023.

16 附录 A 实验代码本附录收录了本文全部五项实验及可视化绘图程序的完整 Python 源代码。所有代码均基于 PyTorch 2.11.0、Transformers 5.8.0、PEFT 0.19.1 及 Scikit-learn 1.8.0 编写, 运行环境为 Ubuntu 22.04 (Python 3.12, CUDA 12.8)。

代码文件包括:

1.01_svm.py——SVM + TF-IDF 基线模型 2.02_bilstm.py——Bi-LSTM 序列分类模型 3.03_bert.py——BERT-base 预训练微调模型 4.04_llm_inference.py——Qwen2.5- 7B 零样本推理 5.05_llm_lora.py——Qwen2.5-7B LoRA 微调模型 6.plot_results.py—— 论文图表可视化脚本各文件均已嵌入早停机制 (Early Stopping), 训练轮数与耗时记录均为实际实验结果。

附 A1 SVM + TF-IDF 基线模型 1""2 01_svm.py 3 优化: max_features 提升至 10000, 充分利用 9.6k+1.2k 文本的丰富词汇。

4""5 import time 6 import pandas as pd 7 import jieba 8 from sklearn.feature_extraction .text import TfidfVectorizer 9 from sklearn.svm import LinearSVC 10 from sklearn.metrics import accuracy_score, precision_recall_fscore_support 11 12 CONFIG = {"train_path": "/root/autodl-tmp/dataset/train-00000-of-

```

00001-02f200ca5f2a7868.parquet",14"dev_path": "/root/autodl-tmp/dataset/validation-00000-of-00001-405
befbaa3bcf1a2.parquet",15"test_path": "/root/autodl-tmp/dataset/test-00000-of-00001-5372924f059fe767.
parquet",16"log_file": "/01_svm_results.txt"17}18 19 def load_and_cut(path):
    20 df = pd.read_parquet(path)1721 df['cut_text'] = df['text'].apply(lambda x: "".join(jieba.cut(str(x))))22 return
df 23 24 if __name__ == "__main__":
    25 print("加载数据与分词中...")26 train_df, dev_df, test_df = load_and_cut(CONFIG["train_path"]),
load_and_cut(CONFIG["dev_path"]), load_and_cut(CONFIG["test_path"])27 28 full_train_df = pd.concat([train_df,
dev_df], ignore_index=True)29 30 print("训练 SVM 中...")31 t0 = time.time()32#优化点: 特征数翻倍 33 tfidf =
TfidfVectorizer(max_features=10000)34 X_train = tfidf.fit_transform(full_train_df['cut_text'])35 36#优化点: 加入
正则化参数 C 调整容错率 37 model = LinearSVC(C=1.0, max_iter=2000)38 model.fit(X_train,
full_train_df['label'])39 train_time = time.time() - t0 40 41 print("测试 SVM 中...")42 t1 = time.time()43 y_pred =
model.predict(tfidf.transform(test_df['cut_text']))44 test_time = time.time() - t1 45 46 acc =
accuracy_score(test_df['label'], y_pred)47 p, r, f1, _ = precision_recall_fscore_support(test_df['label'],
y_pred,average='binary')48 49 log_text = (f"===01.SVM(Optimized Baseline)===\n"50 f"训练数据:
{len(full_train_df)}测试数据: {len(test_df)}\n"51 f"训练耗时: {train_time:.2f}s|测试耗时: {test_time:.2f}s\n"52
f"Accuracy: {acc:.4f}|Precision: {p:.4f}|Recall: {r:.4f}|F1-Score: {f1:.4f}\n\n")53 print(log_text)54 with
open(CONFIG["log_file"], "a", encoding="utf-8") as f: f.write(log_text)Listing 1 01_svm.py—SVM + TF-IDF 基线
模型 18 附 A2 Bi-LSTM 序列分类模型 1""""2 02_bilstm.py 3 优化: 加入 EarlyStopping, 收敛/过拟合时自动停
止, 确保训练时间公平可比。

```

```

4""""5 import time 6 import pandas as pd 7 import jieba 8 import torch 9 import torch.nn as nn 10 from
torch.utils.data import Dataset, DataLoader 11 from collections import Counter 12 from sklearn.metrics import
accuracy_score, precision_recall_fscore_support 13 14 CONFIG = {15"train_path": "/root/autodl-
tmp/dataset/train-00000-of-00001-02f200ca5f2a7868.parquet",16"dev_path": "/root/autodl-
tmp/dataset/validation-00000-of-00001-405 befbaa3bcf1a2.parquet",17"test_path": "/root/autodl-
tmp/dataset/test-00000-of-00001-5372924f059fe767.
parquet",18"log_file": "/02_bilstm_results.txt",19"max_length": 128,"vocab_size": 15000,"embed_dim":
128,"hidden_dim": 128,20"batch_size": 64,"max_epochs": 50,"lr": 1e-3,"early_stop_patience": 5,21"device":
"cuda"if torch.cuda.is_available()else"cpu"22}23 24 class ChnSentiDataset(Dataset):
    25 def __init__(self, df, vocab, max_len):
        26 self.labels = torch.tensor(df['label'].values, dtype=torch.long)27 self.texts = torch.tensor([[vocab.get(w,
1)for w in list(jieba.cut(str(t)))][: max_len] + [0]*max(0, max_len-len(list(jieba.cut(str(t)))))]for t in df['text']],
dtype=torch.long)28 def __len__(self): return len(self.labels)29 def __getitem__(self, idx): return self.texts[idx],
self.labels[idx]30 31 class BiLSTMClassifier(nn.Module):
    32 def __init__(self, vocab_size, embed_dim, hidden_dim):
        33 super().__init__()34 self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)1935
self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)36 self.dropout =
nn.Dropout(0.5)#优化点: 50%的神经元随机失活 37 self.fc = nn.Linear(hidden_dim * 2, 2)38 def forward(self,
x):
    39 embedded = self.embedding(x)40_, (hidden, _) = self.lstm(embedded)41 out = torch.cat((hidden[-2,:],
hidden[-1,:]), dim=1)42 return self.fc(self.dropout(out))#在全连接层前使用 Dropout 43 44 class EarlyStopping:
    45""""早停机机制: 验证集 loss 连续 patience 轮不下降则停止""""46 def __init__(self, patience=5, min_delta=0.0):

```

```

47 self.patience = patience 48 self.min_delta = min_delta 49 self.counter = 0 50 self.best_loss = float("inf") 51
self.early_stop = False 52 def __call__(self, val_loss):
53 if val_loss < self.best_loss - self.min_delta:
54 self.best_loss = val_loss 55 self.counter = 0 56 else:
57 self.counter += 1 58 if self.counter >= self.patience:
59 self.early_stop = True 60 return self.early_stop 61 62 if __name__ == "__main__":
63 train_df, dev_df, test_df = pd.read_parquet(CONFIG["train_path"]), pd.
read_parquet(CONFIG["dev_path"]), pd.read_parquet(CONFIG["test_path"]) 64 65 counter = Counter() 66 for
text in train_df['text']: counter.update(jieba.cut(str(text))) 67 vocab = {w: i+2 for i, (w, _) in
enumerate(counter.most_common(CONFIG["vocab_size"]))} 68 vocab['<PAD>'], vocab['<UNK>'] = 0, 1 69 70
train_loader = DataLoader(ChnSentiDataset(train_df, vocab, CONFIG["max_length"]),
batch_size=CONFIG["batch_size"], shuffle=True) 71 dev_loader = DataLoader(ChnSentiDataset(dev_df, vocab,
CONFIG["max_length"]), batch_size=CONFIG["batch_size"]) 72 test_loader =
DataLoader(ChnSentiDataset(test_df, vocab, CONFIG["max_length"]), batch_size=CONFIG["batch_size"]) 73 74
model = BiLSTMClassifier(len(vocab), CONFIG["embed_dim"], CONFIG["hidden_dim"]).to(CONFIG["device"]) 75
criterion = nn.CrossEntropyLoss() 76 optimizer = torch.optim.Adam(model.parameters(), lr=CONFIG["lr"]) 77 78
best_val_acc = 0.0 79 best_model_state = None 80 early_stopper =
EarlyStopping(patience=CONFIG["early_stop_patience"]) 81 82 t0 = time.time() 83 for epoch in
range(CONFIG["max_epochs"]):
84 model.train() 85 train_loss = 0.0 86 for texts, labels in train_loader:
87 optimizer.zero_grad() 88 outputs = model(texts.to(CONFIG["device"])) 89 loss = criterion(outputs,
labels.to(CONFIG["device"])) 90 loss.backward() 91 optimizer.step() 92 train_loss += loss.item() 93 94
model.eval() 95 val_preds, val_labels = [], [] 96 val_loss = 0.0 97 with torch.no_grad():
98 for texts, labels in dev_loader:
99 outputs = model(texts.to(CONFIG["device"])) 100 val_loss += criterion(outputs,
labels.to(CONFIG["device"])).item() 101 val_preds.extend(torch.argmax(outputs, dim=1).cpu().numpy()) 102
val_labels.extend(labels.numpy()) 103 val_acc = accuracy_score(val_labels, val_preds) 104 avg_val_loss = val_loss
/len(dev_loader) 105 print(f"Epoch{epoch+1: 2d}|Train Loss: {train_loss/len(train_loader): .4f}|Val Loss:
{avg_val_loss: .4f}|Val Acc: {val_acc: .4f}") 106 107 if val_acc > best_val_acc:
108 best_val_acc = val_acc 21 109 best_model_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()} 110 111 if early_stopper(avg_val_loss):
112 print(f"EarlyStopping triggered at epoch{epoch+1}(best val acc: {best_val_acc: .4f}") 113 break 114
actual_epochs = epoch + 1 115 train_time = time.time() - t0 116 117
model.load_state_dict(best_model_state) 118 model.eval() 119 test_preds, test_labels = [], [] 120 t1 =
time.time() 121 with torch.no_grad():
122 for texts, labels in test_loader:
123 test_preds.extend(torch.argmax(model(texts.to(CONFIG["device"])), dim=1).cpu().numpy()) 124
test_labels.extend(labels.numpy()) 125 test_time = time.time() - t1 126 127 acc = accuracy_score(test_labels,
test_preds) 128 p, r, f1, _ = precision_recall_fscore_support(test_labels, test_preds, average='binary') 129 log_text
= (f"===02.Bi-LSTM(Best Val Acc: {best_val_acc: .4f}, Epochs: {actual_epochs})===\n" 130 f"训练耗时:
{train_time: .2f}s|测试耗时: {test_time: .2f}s\n" 131 f"Accuracy: {acc: .4f}|Precision: {p: .4f}|Recall: {r: .4f}|F1:

```

{f1: .4f}\n\n")132 print(log_text)133 with open(CONFIG["log_file"], "a", encoding="utf-8") as f:
f.write(log_text)Listing 2 02_bilstm.py—Bi-LSTM 序列分类模型附 A3 BERT -base 预训练微调模型 1""2
03_bert.py 3 优化：加入 EarlyStopping，收敛/过拟合时自动停止，确保训练时间公平可比。

4""5 import time 226 import pandas as pd 7 from datasets import Dataset 8 from transformers import
BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments, EarlyStoppingCallback 9 from
sklearn.metrics import accuracy_score, precision_recall_fscore_support 10 11 CONFIG = {12"train_path":
"/root/autodl-tmp/dataset/train-00000-of-00001-02f200ca5f2a7868.parquet",13"dev_path":
"/root/autodl-tmp/dataset/validation-00000-of-00001-405 bcfbaa3bcf1a2.parquet",14"test_path":
"/root/autodl-tmp/dataset/test-00000-of-00001-5372924f059fe767.

parquet",15"model_local_path": "/root/autodl-tmp/model_origin/bert",16"log_file":
"/.03_bert_results.txt",17"max_length": 128,"batch_size": 32,"max_epochs": 20,"lr": 2e-5,"early_stop_patience": 3
18}19 20 def compute_metrics(pred):

21 labels, preds = pred.label_ids, pred.predictions.argmax(-1)22 p, r, f1, _ =
precision_recall_fscore_support(labels, preds, average='binary')23 return{'accuracy': accuracy_score(labels,
preds),'precision': p,'recall':

r,'f1': f1}24 25 if __name__ == "__main__":

26 tokenizer = BertTokenizer.from_pretrained(CONFIG["model_local_path"], local_files_only=True)27 model
= BertForSequenceClassification.from_pretrained(CONFIG["model_local_path"], num_labels=2,
local_files_only=True)28 tokenize_func = lambda e: tokenizer(e["text"], padding="max_length", truncation=True,
max_length=CONFIG["max_length"])29 train_ds = Dataset.from_pandas
(pd.read_parquet(CONFIG["train_path"]))[['text', 'label']]30 train_ds.map(tokenize_func, batched=True)31 dev_ds =
Dataset.from_pandas(pd.read_parquet(CONFIG["dev_path"]))[['text', 'label']]32 dev_ds.map(tokenize_func,
batched=True)33 test_ds = Dataset.from_pandas(pd.read_parquet(CONFIG["test_path"]))[['text',
'label']]34 test_ds.map(tokenize_func, batched=True)35 36 training_args = TrainingArguments(2335
output_dir="/bert_outputs", num_train_epochs=CONFIG["max_epochs"],36
per_device_train_batch_size=CONFIG["batch_size"], per_device_eval_batch_size=CONFIG["batch_size"],37
learning_rate=CONFIG["lr"], eval_strategy="epoch", save_strategy="epoch",38 load_best_model_at_end=True,
metric_for_best_model="accuracy",39 warmup_step=0.1, weight_decay=0.01,40 logging_strategy="epoch",
report_to="none")41 42 43 trainer = Trainer(44 model=model, args=training_args, train_dataset=train_ds,
eval_dataset=dev_ds,45 compute_metrics=compute_metrics,46 callbacks=[EarlyStoppingCallback(early_
stopping_patience=CONFIG["early_stop_patience"])]47 48 49 t0 = time.time()50 trainer.train()51 train_time =
time.time() - t0 52 53 t1 = time.time()54 metrics = trainer.predict(test_ds).metrics 55 test_time = time.time() -
t1 56 57# 【核心修改区】：提取 Trainer 自动生成的 test_前缀指标，并统一排版 58 actual_epochs
=int(trainer.state .epoch)if trainer.state.epoch is not None else CONFIG["max_epochs"]59 log_text =
(f"==03.BERT Fine-Tuning(Epochs: {actual_epochs})==\n"60 f"训练耗时: {train_time: .2f}s|测试耗时:
{test_time: .2f}s\n"61 f"Accuracy: {metrics['test_accuracy']: .4f}|Precision: {metrics['test_precision']: .4f}|Recall:
{metrics['test_recall']: .4f}|F1-Score: {metrics['test_f1']: .4f}\n\n")62 print(log_text)63 with open(CONFIG["log_file"],
"a", encoding="utf-8") as f: f.write(log_text)Listing 3 03_bert.py—BERT-base 预训练微调模型附 A4 Qwen2. 5-
7B 零样本推理 1""242 04_llm_inference.py 3 任务：在 1.2k test 数据集上进行 0/1 推理。

4""5 import time 6 import pandas as pd 7 import torch 8 from tqdm import tqdm 9 from transformers
import AutoTokenizer, AutoModelForCausalLM 10 from sklearn.metrics import accuracy_score,

```
precision_recall_fscore_support 11 12 CONFIG = {13"test_path": "/root/autodl-tmp/dataset/test-00000-of-00001-5372924f059fe767.
```

```
parquet",14"model_local_path": "/root/autodl-tmp/model_origin/qwen",15"log_file":
"/.04_qwen_zeroshot_results.txt",16"device": "cuda"if torch.cuda.is_available()else"cpu"17}18 19 def
extract_label(text):
```

```
20 if'1'in text or'正'in text: return 1 21 if'0'in text or'负'in text: return 0 22 return 0 23 24 if__name__
=="__main__":
```

```
25 test_df = pd.read_parquet(CONFIG["test_path"])26 y_true, y_pred = test_df['label'].tolist(), [27 28
tokenizer = AutoTokenizer.from_pretrained(CONFIG["model_local_path"], local_files_only=True)29 model =
AutoModelForCausalLM.from_pretrained(30 CONFIG["model_local_path"], device_map="auto",
torch_dtype=torch.bfloat16, local_files_only=True 31)32 model.eval()33 34 total_tokens, t0 = 0, time.time()35
with torch.no_grad():
```

```
36 for text in tqdm(test_df['text'], desc=f"Evaluating on{len(test_df)}Test Rows"):
```

```
37 messages = [{"role": "system", "content": "情感分析：好评输出 1，差评输出 0。"}, {"role": "user", "content":
text}]2538 inputs = tokenizer(tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True), return_tensors="pt").to(CONFIG["device"])39 outputs =
model.generate(**inputs, max_new_tokens=5, do_sample=False, pad_token_id=tokenizer.eos_token_id )40
gen_ids = outputs[0][inputs.input_ids.shape[1]: ]41 total_tokens +=len(gen_ids)42
y_pred.append(extract_label(tokenizer.decode(gen_ids, skip_special_tokens=True)))43 test_time = time.time() -
t0 44 45 acc = accuracy_score(y_true, y_pred)46 p, r, f1, _ = precision_recall_fscore_support(y_true, y_pred,
average='binary')47 log_text = (f'==04.Qwen2.5-7B(Zero-shot)===\n"48 f"耗时：{test_time:.2f}s|速度：
{total_tokens/test_time if test_time>0 else 0:.2f}Tokens/s\n"49 f"Accuracy: {acc:.4f}|Precision: {p:.4f}|Recall:
{r:.4f}|F1: {f1:.4f}\n\n")50 print(log_text)51 with open(CONFIG["log_file"], "a", encoding="utf-8") as f:
f.write(log_text)Listing 4 04_llm_inference.py—Qwen2.5-7B 零样本推理附 A5 Qwen2.5-7B LoRA 微调模型 1""""2
05_llm_lora.py 3 优化：加入 EarlyStopping，收敛/过拟合时自动停止，确保训练时间公平可比。
```

```
4""""5 import time 6 import pandas as pd 7 import torch 8 from datasets import Dataset 9 from transformers
import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments, EarlyStoppingCallback
10 from peft import LoraConfig, get_peft_model, TaskType 11 from sklearn.metrics import accuracy_score,
precision_recall_fscore_support 12 13 CONFIG = {14"train_path": "/root/autodl-tmp/dataset/train-00000-of-00001-02f200ca5f2a7868.parquet",2615"dev_path": "/root/autodl-tmp/dataset/validation-00000-of-00001-405befbaa3bcf1a2.parquet",16"test_path": "/root/autodl-tmp/dataset/test-00000-of-00001-5372924f059fe767.
```

```
parquet",17"model_local_path": "/root/autodl-tmp/model_origin/qwen",18"log_file":
"/.05_qwen_lora_results.txt",19"max_length": 128,"batch_size": 16,"max_epochs": 20,"lr": 1e-4,"early_stop_patience": 3 20}21 22 def compute_metrics(pred):
```

```
23 labels, preds = pred.label_ids, pred.predictions.argmax(-1)24 p, r, f1, _ =
precision_recall_fscore_support(labels, preds, average='binary')25 return{'accuracy': accuracy_score(labels,
preds),'precision': p,'recall':
```

```
r,'f1': f1}26 27 if__name__=="__main__":
```

```
28 tokenizer = AutoTokenizer.from_pretrained(CONFIG["model_local_path"],local_files_only=True)29 if
tokenizer.pad_token is None: tokenizer.pad_token = tokenizer.eos_token 30 31 base_model =
```

```

AutoModelForSequenceClassification.from_pretrained(32 CONFIG["model_local_path"], num_labels=2,
device_map="auto", 33 torch_dtype=torch.bfloat16, local_files_only=True 34)35
base_model.config.pad_token_id = tokenizer.pad_token_id 36 37 peft_config =
LoraConfig(task_type=TaskType.SEQ_CLS, r=16, lora_alpha=32,target_modules=["q_proj","v_proj"],
lora_dropout=0.05)38 model = get_peft_model(base_model, peft_config)39 40 tokenize_func =lambda e:
tokenizer(e["text"], padding="max_length", truncation=True, max_length=CONFIG["max_length"])41 train_ds =
Dataset.from_pandas (pd.read_parquet(CONFIG["train_path"])[['text', 'label']]).map(tokenize_func,
batched=True)42 dev_ds = Dataset.from_pandas(pd.read_parquet(CONFIG["dev_path"])[['text',
'label']]).map(tokenize_func, batched=True)43 test_ds =
Dataset.from_pandas(pd.read_parquet(CONFIG["test_path"])[['text', 'label']]).map(tokenize_func,
batched=True)44 45 training_args = TrainingArguments(2746 output_dir="./qwen_lora_outputs",
num_train_epochs=CONFIG["max_epochs"],47 per_device_train_batch_size=CONFIG["batch_size"],
per_device_eval_batch_size=CONFIG["batch_size"],48 gradient_accumulation_steps=2,49
learning_rate=CONFIG["lr"], bf16=True,50 eval_strategy="epoch", save_strategy="epoch",51
load_best_model_at_end=True, metric_for_best_model="accuracy",52 warmup_step=0.05,53 logging_
strategy="epoch", report_to="none"54)55 56 trainer = Trainer(57 model=model, args=training_args,
train_dataset=train_ds, eval_dataset=dev_ds,58 compute_metrics=compute_metrics,59
callbacks=[EarlyStoppingCallback(early_stopping_patience=CONFIG["early_stop_patience"])]60)61 62 t0 =
time.time()63 trainer.train()64 train_time = time.time() - t0 65 66 t1 = time.time()67 metrics = trainer.predict(test
_ds).metrics 68 test_time = time.time() - t1 69 70# 【核心修改区】：提取 Trainer 自动生成的 test_前缀指标，并
统一排版 71 actual_epochs =int(trainer.state.epoch)if trainer.state.epoch is not None else CONFIG["
max_epochs"]72 log_text = (f"==05.Qwen2.5-7B(LoRA, Epochs: {actual_epochs})==\n"73 f"训练耗时：
{train_time: .2f}s| 测试耗时： {test_time: .2f}s\n"74 f"Accuracy: {metrics['test_accuracy']: .4f}|Precision:
{metrics['test_precision']: .4f}|Recall: {metrics['test_recall']: .4f}|F1-Score: {metrics['test_f1']: .4f}\n\n")75
print(log_text)76 with open(CONFIG["log_file"], "a", encoding="utf-8") as f: f.write(log_text)Listing 5
05_llm_lora.py—Qwen2.5- 7B LoRA 微调模型 28 附 A6 论文图表可视化脚本 1#-*-coding: utf-8-*""3 论文可
视化脚本：五模型情感分析对比图表(DPI=500)4 所有图表保存为 PDF 矢量格式，适配 LaTeX 编译 5""6 7 import
matplotlib 8 matplotlib.use("Agg")9 import matplotlib.pyplot as plt 10 import matplotlib.ticker as mticker 11 from
matplotlib.patches import FancyBboxPatch, FancyArrowPatch 12 import numpy as np 13 import os 14 15
output_dir = os.path.join(os.path.dirname(__file__), "figures")16 os.makedirs(output_dir, exist_ok=True)17
18#===== 19# 全局样
式 -500 DPI 出版级渲染 20#=====
=====21 plt.rcParams.update({22"font.family": "sans-serif",23"font.sans-serif":
["SimHei","Microsoft YaHei","Arial","DejaVu Sans"],24"axes.unicode_minus": False,25"mathtext.default":
"regular",26"figure.dpi": 200,27"savefig.dpi": 500,28"savefig.bbox": "tight",29"savefig.pad_inches":
0.08,30"savefig.format": "pdf",31"axes.linewidth": 0.8,32"axes.edgecolor": "#444444",33"xtick.major.width":
0.6,34"ytick.major.width": 0.6,35"xtick.major.size": 3.5,36"ytick.major.size": 3.5,37"xtick.color":
"#333333",38"ytick.color": "#333333",39"axes.titlesize": 13,2940"axes.labelsize": 12,41"legend.fontsize":
10,42"legend.framealpha": 0.85,43"legend.edgecolor": "#cccccc",44"legend.fancybox": True,45"figure.facecolor":
"white",46"axes.facecolor": "#fcfcfc",47})48 49 MODELS_SHORT = ["SVM","Bi-LSTM","BERT", "Qwen\nZero-
shot","Qwen\nLoRA"]50 MODELS_ONELINE = ["SVM","Bi-LSTM", "BERT","Qwen Zero-shot","Qwen LoRA"]51

```

```

52#5 色专业渐变配色( 蓝色→橙色→青色→紫色→朱红)53 C = {54"svm": "#2471A3",#传统统计-深蓝 55"lstm":
"#E67E22",#深度学习-橘色 56"bert": "#27AE60",#预训练最佳-翠绿 57"qwen_zero": "#8E44AD",#零样本-紫色
58"qwen_lora": "#C0392B",#微调-深红 59"best_hl": "#F9E79F",#高亮底色-鹅黄 60}61 COLORS = [C["svm"],
C["lstm"], C["bert"], C["qwen_zero"], C["qwen_lora"]]62 63#--- 原始数据---64 accuracy = [0.8933, 0.8983,
0.9517, 0.9000, 0.9567]65 precision = [0.8974, 0.8945, 0.9568, 0.9251, 0.9633]66 recall = [0.8914, 0.9062, 0.9474,
0.8734, 0.9507]67 f1= [0.8944, 0.9003, 0.9521, 0.8985, 0.9570]68 train_t = [0.32, 4.05, 184.66, 0.1414.10]69 infer_t
= [0.02, 0.02, 1.06, 43.84, 7.19]70 71 best_idx = 4#Qwen-LoRA 在所有指标上最佳 72 73 74 def save(name):
75 path = os.path.join(output_dir, name)76 plt.savefig(path, dpi=500)77 print( f"[OK]{name}")78 plt.close()79
80 81 def style_ax(ax):
82 """" 统一坐标轴样式 """83 ax.spines["top"].set_visible(False) 84 ax.spines["right"].set_visible(False)85
ax.spines["left"].set _color("#555555")86 ax.spines["bottom"].set_color("#555555") 87
ax.tick_params(colors="#444444")88 ax.grid(axis="y", alpha=0.25, linestyle=(0, (3, 4)), color="#aaaaaa",
zorder=0)89 90 91 def labelBars(ax, bars, fmt="{: .4f}", offset=0.004, color="#333333", fontsize=8.5):
92 for bar in bars:
93 h = bar.get_height()94 ax.text(bar.get_x() + bar.get_width() / 2., h + offset,95 fmt.format(h), ha="center",
va="bottom",96 fontsize=fontsize, fontweight="bold", color=color)97 98 99#=====
=====100# Fig D1: 综合性能对比—分组柱状图+
最高亮标注 101#=====
=====102 def fig_d1_performance():
103 fig, ax = plt.subplots(figsize=(8.0, 4.6))104 x = np.arange(len(MODELS_SHORT))105 w = 0.30 106 gap =
0.02 107 108 b1 = ax.bar(x - w/2 - gap, accuracy, w, label="Accuracy", 109 color="#2C3E50", edgecolor="white",
linewidth=0.5, zorder=3)110 b2 = ax.bar(x + w/2 + gap, f1, w, label="F1-Score",111 color=C["bert"],
edgecolor="white", linewidth=0.5, zorder=3)112 113 labelBars(ax, b1, offset=0.002, fontsize=7.5)114
labelBars(ax, b2, offset=0.002, fontsize=7.5)115 116#高亮最佳模型区域 117 ax.axvspan(best_idx - 0.45,
best_idx + 0.45, facecolor=C["best_hl"],118 alpha=0.35, zorder=0)119 ax.annotate("Best", xy=(best_idx, 0.955),
fontsize=9, fontweight="bold",120 color="#8B0000", ha="center", 121 bbox=dict(boxstyle="round,pad=0.3",
facecolor="#fff2cc",31122 edgecolor="#e6c300", alpha =0.9))123 124 ax.set_xticks(x)125
ax.set_xticklabels(MODELS_SHORT, fontsize=10.5)126 ax.set_ylabel("Score", fontsize=12, fontweight=" bold",
color="#2C3E50")127 ax.set_ylim(0.865, 0.965)128
ax.yaxis.set_major_formatter(mticker.FormatStrFormatter("% .3f"))129 ax.legend(loc="lower right", frameon=True,
fontsize=10.5,130 handlelength=1.2, handleheight=0.7)131 style_ax(ax)132 fig .tight_layout()133
save("fig_d1_performance.pdf")134 135 136#=====
=====137# Fig D2: 训练耗时—对数纵轴 + 渐变配色
138#=====
=====139 def
fig_d2_train_time():
140 fig, ax = plt.subplots(figsize=(8.2, 4.6))141 x = np.arange(len(MODELS_SHORT))142 td = [train_t[0],
train_t[1], train_t[2], 0, train_t[4]]143 bars = ax.bar(x, [v if v > 0 else 1e-5 for v in td],144 color=COLORS,
edgecolor="white", linewidth=0.6, zorder=3, width=0.55)145 146 for i, (bar, t)in enumerate(zip(bars, td)):
147 if t == 0:
148 ax.text(bar.get_x() + bar.get_width()/2, 2, 149"Zero-shot\n--", ha="center", va="bottom",150
fontsize=8.5, fontstyle="italic", color="#888888", fontweight="bold")151 else:

```

```

152 offset =max(t * 0.08, 1)153 ax.text(bar.get_x() + bar.get_width()/2, t + offset,154 f"{t: .2f}s", ha="center",
va="bottom",155     fontsize=9,     fontweight="bold",     color="#333333")156 157 ax.set_yscale("log")158
ax.set_xticks(x)159 ax.set_xticklabels(MODELS_SHORT, fontsize=10.5)160 ax.set_ylabel("Training Time(s,log
scale)", fontsize=12, fontweight="bold", color="#2C3E50")32161 ax.set_ylim(0.15, 3500)162 ax.yaxis.set_major_
formatter(mticker.ScalarFormatter())163 style_ax(ax)164 fig.tight_layout()165 save("fig_d2_train_time.pdf")166
167 168#===== 169#
Fig D3: 推理耗时—对数纵轴+效率标签 170#=====
=====171 def fig_d3_infer_time():
172 fig, ax = plt.subplots(figsize=(8.0, 4.4))173 x = np.arange(len(MODELS_SHORT))174 bars = ax.bar(x,
infer_t, color=COLORS, edgecolor="white", linewidth=0.6,175 zorder=3, width=0.55)176 177 for bar, t in
zip(bars, infer_t):
178 offset =max(t * 0.08, 0.3)179 ax.text(bar.get_x() + bar.get_width()/2, t + offset,180 f"{t: .2f}s", ha="center",
va="bottom",181     fontsize=9, fontweight="bold", color="#333333")182 183#效率标签(samples/s)—置于时间数
值上方 184 speed_labels = ["60, 000","60,000","1,132","27","167"]185 for i, (bar, t)in enumerate(zip(bars, infer_t)):
186 time_label_y = t +max(t * 0.08, 0.3)187 ax.text(bar.get_x() + bar.get_width()/2, time_label_y * 1.9,188
f"{speed_labels[i]}samp/s", ha="center", va="bottom",189     fontsize=7,     fontstyle="italic",     color="#666666",
fontweight="bold")190 191 ax.set_yscale("log")192 ax.set_xticks(x)193 ax.set_xticklabels(MODELS_SHORT,
fontsize=10.5)194 ax.set_ylabel("Inference Time(s,log scale)", fontsize=12, fontweight="bold",
color="#2C3E50")195 ax.set_ylim(0.015, 200)196 ax.yaxis.set_major_formatter(mticker.ScalarFormatter())197
style_ax(ax)198 fig.tight_layout()199 save ("fig_d3_infer_time.pdf")200 33201 202#=====
=====203# Fig D4: 精度-效率权衡—
散点 + 气泡 + 帕累托前沿 204#=====
=====205 def fig_d4_tradeoff():
206 fig, ax = plt.subplots(figsize=(8.0, 5.2))207 sizes = [220, 220, 380, 320, 420]208 edge_colors =
["#1a5276", "#ba4a00", "#1e8449", "#6c3483", "#922b21"]209 for i in range(5):
210 tx = train_t[i]if train_t[i] > 0 else 0.8 211 ax.scatter(tx, f1[i], s=sizes[i], c=COLORS[i],212
edgecolors=edge_colors[i], linewidth=1.5, 213 zorder=6, alpha=0.93)214#帕累托前沿连线 215 pareto_idx = [0,
2, 4]216 pareto_x = [train_t[0]if train_t[0] > 0 else 0.8, train_t[2], train_t[4]]217 pareto_y = [f1[0], f1[2], f1[4]]218
ax.plot(pareto_x, pareto_y, color="#777777", linewidth=1.5, 219 linestyle="--", marker="o", markersize=0,
alpha=0.6, zorder=2)220 ax.annotate("Pareto Frontier", xy=(pareto_x[1], pareto_y[1]),221 xytext=(35, -28),
textcoords="offset points", 222     fontsize=8.5,     fontstyle="italic",     color="#666666",223
arrowprops=dict(arrowstyle="->", color="#999999", lw=1.0))224#标注标签 225 offsets = [(15, -15), (20, -18),
(20, -15), (15, -22), (25, -15)]226 for i in range(5):
227 tx_ = train_t[i]if train_t[i] > 0 else 0.8 228 ax.annotate(MODELS_ONELINE[i], (tx_, f1[i]),229
textcoords="offset points", xytext=offsets[i],230     fontsize=9.5,     fontweight="bold",     color=COLORS[i])231
ax.set_xscale("log")232 ax.set_xlabel("Training Time(s, log scale)", fontsize=12,
fontweight="bold",color="#2C3E50")233 ax.set_ylabel("F1-Score", fontsize=12, fontweight="bold",
color="#2C3E50")234 ax.set_ylim (0.878, 0.964)235 ax.set_xlim(0.15, 3800)236 style_ax(ax)237
fig.tight_layout()238 save("fig_d4_ tradeoff.pdf")239 240 241#=====
=====34242# Fig D5: Precision-Recall 对比—
带差异标注 243#=====
=====

```

```

=====244 def fig_d5_precision_recall():
    245 fig, ax = plt.subplots(figsize=(8.0, 4.6))246 x = np.arange(len(MODELS_SHORT))247 w = 0.30 248 gap =
    0.02 249 250 b1 = ax.bar(x - w/2 - gap, precision, w, label="Precision", 251 color="#8E44AD", edgecolor="white",
    linewidth=0.5, zorder=3)252 b2 = ax.bar(x + w/2 + gap, recall, w, label="Recall",253 color="#E67E22",
    edgecolor="white", linewidth=0.5, zorder=3)254 255 label_bars(ax, b1, offset=0.002)256 label_bars(ax, b2,
    offset=0.002)257 258# 绘制 Precision-Recall 差异—置于柱状图顶部数值上方 259 for i in range(5):
    260 diff = precision[i] - recall[i]261 top_y =max(precision[i], recall[i]) + 0.010 262 sign = "+" if diff > 0 else ""263
    ax.annotate(f"P-R={sign}{diff: .4f}", xy=(x[i], top_y),264 fontsize=7, ha="center", va="bottom", 265
    color="#333333",266 fontweight="bold",267 bbox=dict(boxstyle="round,pad=0.2",268 facecolor="ffffff", 269
    edgecolor="#888888", 270 alpha=0.9, linewidth=0.5))271 272 ax.set_xticks(x)273
    ax.set_xticklabels(MODELS_SHORT, fontsize=10.5)274 ax.set_ylabel("Score", fontsize=12, fontweight=" bold",
    color="#2C3E50")275 ax.set_ylim(0.845, 0.978)276
    ax.yaxis.set_major_formatter(mticker.FormatStrFormatter("%.3f"))277 ax.legend(loc="lower right", frameon=True,
    fontsize=10.5,278 handlelength=1.2, handleheight=0.7)279 fig.tight_layout() 280
    save("fig_d5_precision_recall.pdf")281 282 283#=====
    =====35284# Fig D6: F1 性能步进图 (重设计—绝对 F1+ 增量标
    注)285#=====
    =====286 def
    fig_d6_waterfall():
    287 fig, ax = plt.subplots(figsize=(7.8, 4.6))288 x = np.arange(len(MODELS_SHORT))289 290 bars = ax.bar(x,
    f1, color=COLORS, edgecolor="white", linewidth=0.8,291 width=0.50, zorder=3)292 293#标注绝对 F1 值 294
    for i, bar in enumerate(bars):
    295 ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.002,296 f"{f1[i]: .4f}", ha="center",
    va="bottom",297 fontsize=10, fontweight="bold", color="#1a1a1a" )298 299#相邻模型间的变化箭头与增量值
    300 for i in range(len(f1) - 1):
    301 mid_x = (x[i] + x[i+1]) / 2 302 mid_y = (f1[i] + f1[i+1]) / 2 303 delta = f1[i+1] - f1[i]304 sign = "+" if delta >
    0 else ""305 color_d = "#1a7a1a" if delta > 0 else "#b71c1c"306#连接线 307 ax.plot([x[i] + 0.28, x[i+1] - 0.28],
    [f1[i], f1[i+1]],308 color="#777777", linewidth=1.0, linestyle="--", alpha=0.7, zorder=2)309#增量标签 310
    ax.annotate(f"{sign}{delta: .4f}", xy=(mid_x, mid_y),311 fontsize=8.5, fontweight="bold", color=color_d,312
    ha="center", va="center",313 bbox=dict(boxstyle="round,pad=0.2", facecolor="white",314 edgecolor=color_d,
    alpha=0.9, linewidth=0.7))315 316#基线 317 ax.axhline(y=f1[0], color=C["svm"], linewidth=1.0, linestyle="--
    ",318 alpha=0.5, zorder=1)319 ax.text(4.2, f1[0] + 0.001, f"SVM baseline({f1[0]: .4f})",320 fontsize=8,
    color=C["svm"], fontstyle="italic", alpha=0.8)321 322 ax.set_xticks(x)323 ax.set_xticklabels(MODELS_SHORT,
    fontsize=10.5)324 ax.set_ylabel("F1-Score", fontsize=12, fontweight="bold", color="#2C3E50")36325
    ax.set_ylim(0.875, 0.960)326 style_ax(ax)327 fig.tight_layout()328 save("fig_d6_ waterfall.pdf")329 330
    331#=====
    =====332# Fig
    D7(新增): 雷达图—五模型四维归一化 333#=====
    =====334 def fig_d7_radar():
    335 categories = ["Accuracy", "Precision", "Recall", "F1-Score"]336 N =len(categories)337#归一化到 0.88~0.95
    可视范围 338 data = np.array([339[0.8933, 0.8974, 0.8914, 0.8944],340[0.8983, 0.8945, 0.9062, 0.9003],
    341[0.9517, 0.9568, 0.9474, 0.9521],342[0.9000, 0.9251, 0.8734, 0.8985], 343[0.9567, 0.9633, 0.9507,
    0.9570],344])345 data_norm = (data - 0.87) / (0.97 - 0.87)346 347 angles = np.linspace(0, 2 * np.pi, N,

```

```

endpoint=False).tolist()348 angles += angles[: 1]349 350 fig, ax = plt.subplots(figsize=(7.0, 7.0),
subplot_kw=dict(polar=True))351 ax.set_theta_offset(np.pi / 2)352 ax .set_theta_direction(-1)353 354 for i in
range(5):
    355 vals = data_norm[i].tolist() + data_norm[i].tolist()[: 1]356 ax.fill(angles, vals, alpha=0.08,
color=COLORS[i])357 ax.plot(angles, vals,"o-", linewidth=1.3, color=COLORS[i],358 markersize=5,
label=MODELS_ONELINE[i])359 360 ax.set_xticks(angles[: -1])361 ax.set_xticklabels(categories, fontsize=10.5,
fontweight="bold")362 ax.set_ylim(0, 1.15)363 ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])364 ax.set_yticklabels(["0.890",
"0.910","0.930","0.950","0.970"], fontsize=7.5, color="#888888")365 ax.legend(loc="upper right",
bbox_to_anchor=(1.35, 1.12), 37366 frameon=True, fontsize=9.5, ncol=1)367 ax.grid(True, alpha=0.3,
linestyle="--")368 fig.tight_layout()369 save("fig_d7_radar.pdf" )370 371 372#=====
=====373# Fig D8(新增): 训练/推理时间双轴对比
374#=====
=====375 def
fig_d8_dual_axis_time():
    376 fig, ax1 = plt.subplots(figsize=(8.2, 4.8))377 x = np.arange(len(MODELS_SHORT))378 w = 0.35 379 380
td_plot = [train_t[0], train_t[1], train_t[2], 0.5, train_t[4]]381 b1 = ax1.bar(x - w/2, td_plot, w, label="Training
Time(s)",382 color="#3498DB", edgecolor="white", linewidth=0.5, zorder=3)383 ax1.set_ylabel("Training
Time(s)", fontsize=11, fontweight="bold", color=" #2471A3")384 ax1.set_yscale("log")385 ax1.set_ylim(0.15,
3500)386 ax1.tick_params(axis="y", colors="#2471A3")387 388 ax2 = ax1. twinx()389 b2 = ax2.bar(x + w/2,
infer_t, w, label="Inference Time(s)",390 color="#E74C3C", edgecolor="white", linewidth=0.5, zorder=3)391
ax2.set_ylabel("Inference Time(s)", fontsize=11, fontweight="bold", color=" #C0392B")392
ax2.set_yscale("log")393 ax2.set_ylim(0.01, 200)394 ax2.tick_params(axis="y", colors="#C0392B")395 396#标注
397 for i, (t_r, t_i)in enumerate(zip(td_plot, infer_t)):
    398 ax1.text(x[i] - w/2, t_r * 1.4 if t_r > 0 else 0.8, f'{train_t[i]: .1f}s",399 ha="center", fontsize=7.5,
fontweight="bold", color="#2471A3")400 ax2.text(x[i] + w/2, t_i * 1.3, f'{infer_t[i]: .2f}s",401 ha="center",
fontsize=7.5, fontweight="bold", color="#C0392B")402 403 ax1.set_xticks(x)404
ax1.set_xticklabels(MODELS_SHORT, fontsize=10)405 ax1.spines["top"].set_visible(False)38406 ax2.spines
["top"].set_visible(False)407 ax1.grid(axis="y", alpha=0.2, linestyle=(0, (3, 4)), color="#aaaaaa", zorder=0) 408
fig.tight_layout()409 save("fig_d8_dual_time.pdf")410 411 412# =====
=====413# Fig D9(新增): 模型规模 - 性能气泡图
414#=====
=====415 def
fig_d9_model_size():
    416 fig, ax = plt.subplots(figsize=(8.0, 4.8))417 params = [0.0, 2.0, 110, 7000, 7000 + 3.4]#百万 418 markers
= [180, 200, 380, 550, 580]419 420 for i in range(5):
    421 ax.scatter(params[i], f1[i], s=markers[i], c=COLORS[i],422 edgecolors="#333333", linewidth=1.0,
zorder=5, alpha=0.9)423 424 for i in range(5):
    425 offset = (15, 0.001)if i != 3 else (-35, -0.003)426 ax.annotate(MODELS_ONELINE[i], (params[i], f1[i]),427
textcoords="offset points", xytext=offset,428 fontsize=9, fontweight="bold", color=COLORS[i])429 430
ax.set_xscale("symlog", linthresh=1)431 ax.set_xlabel("Model Parameters(Millions,log scale)", fontsize=11,432
fontweight="bold", color="#2C3E50")433 ax.set_ylabel("F1-Score", fontsize=11, fontweight="bold",
color="#2C3E50")434 ax.set_ylim(0.882, 0.964)435 ax.set_xlim(left=-0.5, right=11000)436 style_ax(ax)437
fig.tight_layout()438 save("fig_d9_model_size.pdf")439 440 441 if __name__ == "__main__":

```

```
442 fig_d1_performance()443 fig_d2_train_time()444 fig_d3_infer_time()445 fig_d4_tradeoff()446
fig_d5_precision_recall()39447 fig_d6_waterfall()448 fig_d7_radar()449 fig_d8_dual_axis_time()450 fig
_d9_model_size()451 print(f"\nAll 9 figures saved to: {output_dir}")452 print("DPI: 500|Format: PDF Vector")Listing
6 plot_results.py—论文图表可视化脚本 40
```